The Oil Painting Effect

As part of the Math 155 Project (Spring 2023)

```
clear;clc;close all;
```

Algorithm Implementation

The classic approach to creating the oil painting effect is to replace all pixel values of a mxn matrix with the most frequently occuring value.

```
% Read in the input image as f
f = imread("/Users/xiangdilin/Desktop/math 155/photo.jpg");
f = double(f);
imshow(uint8(f));
```



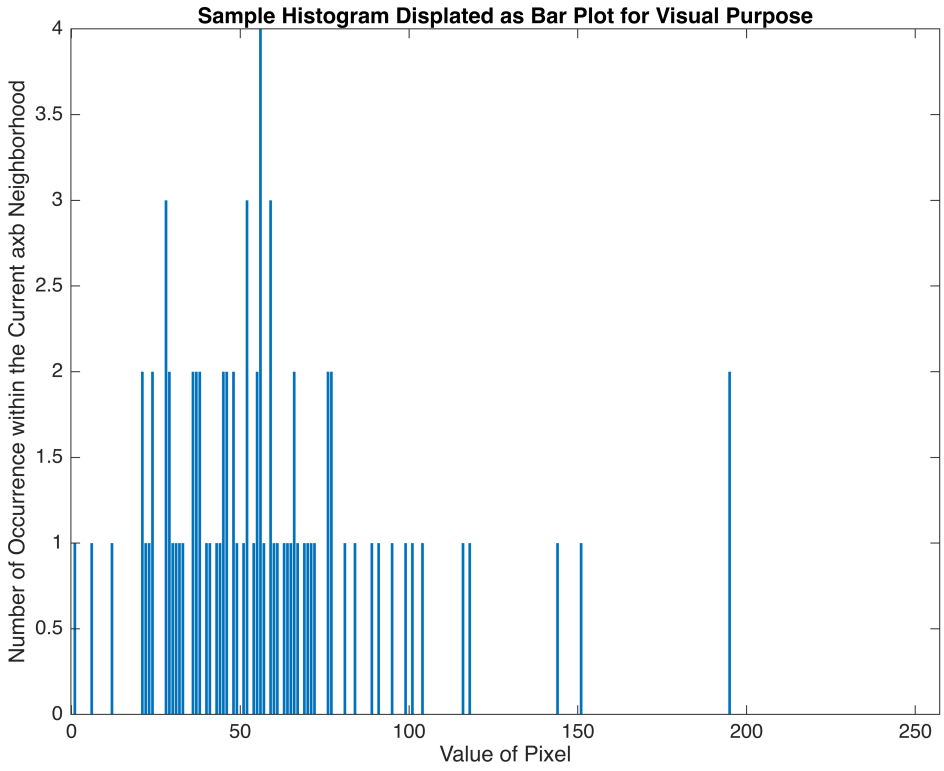
```
a = 10; b = 8; % Define the matrix size of own choice
g = zeros([size(f,1)-a,size(f,2)-b,3]);

% Iterate over f to calculate the histogram for each axb neighborhood
for n=1:3 % Identifier of the RGB type
    for i=1:size(f,1)-a
        for j=1:size(f,2)-b
            nbhd = f(i:i+a-1,j:j+b-1,n);
            h = zeros(1,256);
            for x=1:(a*b)
```

```
                h(nbhd(x)+1) = h(nbhd(x)+1)+1;
            end
            [maxval,pos] = max(h); % Find the most frequently occuring
pixel value
            g(i,j,n) = pos-1;
        end
    end
end
```

The following cell produces a sample histogram generated during the rum time of the algorithm above for a certain color (R/G/B) when evaluating each axb region.

```
figure
bar(h)
title("Sample Histogram Displated as Bar Plot for Visual Purpose")
xlabel("Value of Pixel")
ylabel("Number of Occurrence within the Current axb Neighborhood")
```




```
% Display the output Image
imshow(uint8(g));
```



Experimentation with Different Parameter Values

Here same algorithm is implemented but with different choice of a&b values.

```
% Try using different pair of a&b values
a = 8; b = 6; % Define the matrix size of own choice
g1 = zeros([size(f,1)-a,size(f,2)-b,3]);

% Iterate over f to calculate the histogram for each axb neighborhood
for n=1:3 % Identifier of the RGB type
    for i=1:size(f,1)-a
        for j=1:size(f,2)-b
            nbhd = f(i:i+a-1,j:j+b-1,n);
            h = zeros(1,256);
            for x=1:(a*b)
                h(nbhd(x)+1) = h(nbhd(x)+1)+1;
            end
            [maxval,pos] = max(h); % Find the most frequently occuring
pixel value
            g1(i,j,n) = pos-1;
        end
    end
end
```

```
% Display all the images
figure
```

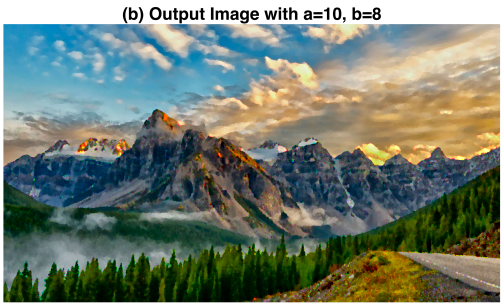
```
set(gcf, "Position", [100 100 7000 4000])

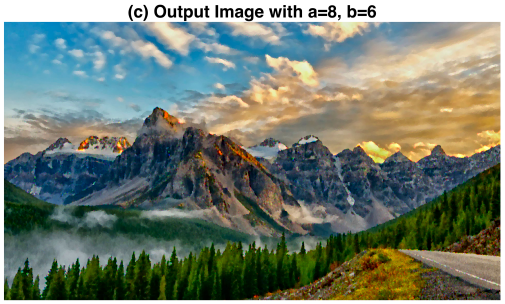
subplot(2,2,1)
imshow(uint8(f))
title('(a) Original Image')

subplot(2,2,2)
imshow(uint8(g))
title('(b) Output Image with a=10, b=8')

subplot(2,2,3)
imshow(uint8(g1))
title('(c) Output Image with a=8, b=6')
```







Performance Varies on Different Input Images

The algorithm works well for rich-colored images like the landscape photo used above. However, it does not ideally process certain types of inputs, which are usually characterized by sharp edges and uni-color regions. The following palette image is an example where the oil painting will not properly apply to.

```
f = imread("/Users/xiangdilin/Desktop/math 155/palette.png");
f = double(f);

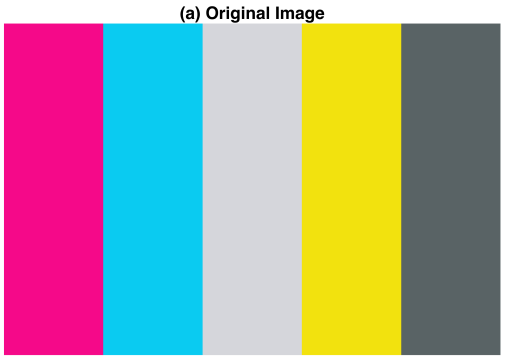
a = 8; b = 6; % Define the matrix size of own choice
g = zeros([size(f,1)-a,size(f,2)-b,3]);
```

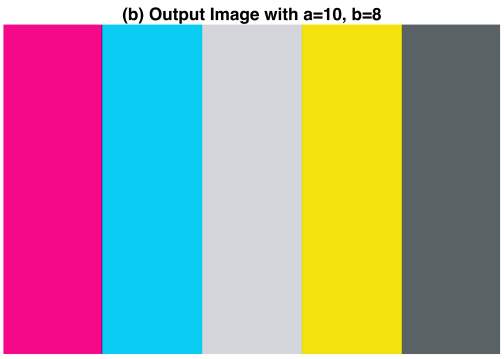
```
% Iterate over f to calculate the histogram for each axb neighborhood
for n=1:3 % Identifier of the RGB type
    for i=1:size(f,1)-a
        for j=1:size(f,2)-b
            nbhd = f(i:i+a-1,j:j+b-1,n);
            h = zeros(1,256);
            for x=1:(a*b)
                h(nbhd(x)+1) = h(nbhd(x)+1)+1;
            end
            [maxval,pos] = max(h); % Find the most frequently occuring
pixel value
            g(i,j,n) = pos-1;
        end
    end
end
```

```
% Display all the images
figure
set(gcf, "Position", [100 100 6000 2000])

subplot(1,2,1)
imshow(uint8(f))
title('(a) Original Image')

subplot(1,2,2)
imshow(uint8(g))
title('(b) Output Image with a=10, b=8')
```





As shown in the result, no observable changes have taken place in the output. The limitations of oil painting effect algorithm have inspired us to explore the possible enhancements, and we found the Van Gogh's stroke style algorithm to be a great extension of it which may overcome the setbacks.